

Activity - CIA 2

Name: Avdhut kale

Roll no: 43

✓ Boltzmann Machine for Customer Categorization

Assignment: Categorize customer segments based on their shopping habits

This notebook contains the implementation of a Boltzmann machine model to categorize customer segments based on their shopping habits using the Online Retail II dataset.

Tasks:

1. Load the Online Retail II dataset and clean the data by removing incomplete entries.
2. Preprocess the data by encoding categorical data and scaling numerical features.
3. Transform the data into a binary format (purchased or not purchased).
4. Train a Boltzmann machine (Restricted Boltzmann Machine) to learn the underlying probability distribution.

Upload Dataset

Please upload the `online_retail_II-1.csv` file using the button below.

✓ Task 1: Load and Clean Data

In this task, we will load the dataset and remove any incomplete entries, such as missing customer IDs, which are essential for our user-based analysis. We will also remove cancelled orders and rows with invalid quantities or prices.

```
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

# 1. Load the dataset
# Update the path to point to your specific dataset location if running locally
# file_path is already set to 'online_retail_II-1.csv' from the previous cell
print("Loading dataset...")
df = pd.read_excel('online_retail.xlsx')

print(f"Original dataset shape: {df.shape}")

# 2. Clean the data
# Remove rows with missing Customer ID, as we need it to identify customer shopping habits
df_clean = df.dropna(subset=['Customer ID'])

# Remove rows with missing Description
df_clean = df_clean.dropna(subset=['Description'])

# Remove cancelled orders (Invoices starting with 'C')
df_clean = df_clean[~df_clean['Invoice'].astype(str).str.startswith('C')]

# Keep only valid quantities and prices (greater than 0)
df_clean = df_clean[(df_clean['Quantity'] > 0) & (df_clean['Price'] > 0)]

print(f"Cleaned dataset shape: {df_clean.shape}")
df_clean.head()
```

Loading dataset...
 Original dataset shape: (525461, 8)
 Cleaned dataset shape: (407664, 8)

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.0	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom

Task 2: Preprocess Data

We will preprocess the data by encoding categorical data and scaling numerical features to normalize the range of data values. Although the final model requires binary data for item purchases, standardizing numerical features and encoding categorical features can be useful for additional analysis or downstream tasks.

```
from sklearn.preprocessing import StandardScaler, LabelEncoder

# Copy the cleaned dataset to avoid warnings
df_preprocessed = df_clean.copy()

# Encode Categorical Data
# Encoding 'Country' as an example of categorical encoding
label_encoder = LabelEncoder()
df_preprocessed['Country_Encoded'] = label_encoder.fit_transform(df_preprocessed['Country'])

# Encoding 'StockCode' to have numeric identifiers
df_preprocessed['StockCode_Encoded'] = label_encoder.fit_transform(df_preprocessed['StockCode'].astype(str))

# Scale Numerical Features
# We scale 'Quantity' and 'Price' using StandardScaler
scaler = StandardScaler()
df_preprocessed[['Quantity_Scaled', 'Price_Scaled']] = scaler.fit_transform(df_preprocessed[['Quantity', 'Price']])

print("Preprocessing complete. Sample of encoded and scaled features:")
display(df_preprocessed[['Country', 'Country_Encoded', 'StockCode', 'StockCode_Encoded', 'Quantity', 'Quantity_Scaled', 'Price', 'Price_Scaled']])
```

Preprocessing complete. Sample of encoded and scaled features:

	Country	Country_Encoded	StockCode	StockCode_Encoded	Quantity	Quantity_Scaled	Price	Price_Scaled
0	United Kingdom	34	85048	3410	12	-0.016373	6.95	0.105172
1	United Kingdom	34	79323P	2729	12	-0.016373	6.75	0.099418
2	United Kingdom	34	79323W	2731	12	-0.016373	6.75	0.099418
3	United Kingdom	34	22041	1250	48	0.355372	2.10	-0.034364
4	United Kingdom	34	21232	615	24	0.107542	1.25	-0.058819

Task 3: Transform Data into Binary Format

We need to transform the data into a suitable format where each customer's shopping habits over time are captured in a binary format—purchased (1) or not purchased (0). We will create a user-item matrix where rows represent Customer IDs and columns represent StockCodes.

```
# Create a cross-tabulation of customers and their purchased items (quantities)
# This will result in a matrix where rows are customers, columns are items, and values are quantities
basket = pd.crosstab(df_preprocessed['Customer ID'], df_preprocessed['StockCode_Encoded'])

# Transform the data into binary format: 1 if purchased (quantity > 0), 0 otherwise
# This represents the shopping habit of each customer
basket_binary = (basket > 0).astype(int)
```

```
print(f"Binary User-Item Matrix shape: {basket_binary.shape}")
print(f"Number of Customers: {basket_binary.shape[0]}")
print(f"Number of Unique Items: {basket_binary.shape[1]}")

# Display a small subset of the sparse matrix
display(basket_binary.iloc[:5, :10])
```

```
Binary User-Item Matrix shape: (4312, 4017)
Number of Customers: 4312
Number of Unique Items: 4017
```

StockCode_Encoded	0	1	2	3	4	5	6	7	8	9
Customer ID										
12346.0	0	0	0	0	0	0	0	0	0	0
12347.0	0	0	0	0	0	0	0	0	0	0
12348.0	0	0	0	0	0	0	0	0	0	0
12349.0	0	0	0	0	0	0	0	0	0	0
12351.0	0	0	0	0	0	0	0	0	0	0

Task 4: Train Boltzmann Machine

We will train a Restricted Boltzmann Machine (RBM) using the binary user-item matrix. RBMs are unsupervised learning models capable of learning the underlying probability distribution of the data. In the context of retail, the RBM learns which items are frequently bought together, capturing the complex dependencies between purchased items for different customer segments.

```
from sklearn.neural_network import BernoulliRBM
from sklearn.model_selection import train_test_split

# Convert the dataframe to a numpy array for training
X = basket_binary.values

# Split the data into training and testing sets to evaluate later if needed
# We will primarily train on the training set as requested
X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)

print(f"Training set size: {X_train.shape}")
print(f"Testing set size: {X_test.shape}")

# Initialize the Restricted Boltzmann Machine
# n_components: number of hidden units (latent features/customer segments)
# learning_rate: step size for weight updates
# n_iter: number of passes over the training data
rbm = BernoulliRBM(n_components=50, learning_rate=0.01, n_iter=20, random_state=42, verbose=True)

# Train the Boltzmann machine
print("Training the Restricted Boltzmann Machine...")
rbm.fit(X_train)

print("Training completed.")
```

```
Training set size: (3449, 4017)
Testing set size: (863, 4017)
Training the Restricted Boltzmann Machine...
[BernoulliRBM] Iteration 1, pseudo-likelihood = -972.91, time = 4.20s
[BernoulliRBM] Iteration 2, pseudo-likelihood = -575.89, time = 2.28s
[BernoulliRBM] Iteration 3, pseudo-likelihood = -408.08, time = 2.20s
[BernoulliRBM] Iteration 4, pseudo-likelihood = -316.20, time = 2.22s
[BernoulliRBM] Iteration 5, pseudo-likelihood = -308.47, time = 2.43s
[BernoulliRBM] Iteration 6, pseudo-likelihood = -307.76, time = 4.80s
[BernoulliRBM] Iteration 7, pseudo-likelihood = -306.61, time = 2.17s
[BernoulliRBM] Iteration 8, pseudo-likelihood = -303.44, time = 2.16s
[BernoulliRBM] Iteration 9, pseudo-likelihood = -297.59, time = 2.18s
[BernoulliRBM] Iteration 10, pseudo-likelihood = -282.93, time = 2.14s
[BernoulliRBM] Iteration 11, pseudo-likelihood = -278.24, time = 4.99s
[BernoulliRBM] Iteration 12, pseudo-likelihood = -272.63, time = 2.17s
[BernoulliRBM] Iteration 13, pseudo-likelihood = -272.36, time = 2.18s
[BernoulliRBM] Iteration 14, pseudo-likelihood = -270.09, time = 2.21s
```

```
[BernoulliRBM] Iteration 15, pseudo-likelihood = -268.29, time = 2.21s
[BernoulliRBM] Iteration 16, pseudo-likelihood = -267.30, time = 5.14s
[BernoulliRBM] Iteration 17, pseudo-likelihood = -266.49, time = 2.21s
[BernoulliRBM] Iteration 18, pseudo-likelihood = -267.26, time = 2.34s
[BernoulliRBM] Iteration 19, pseudo-likelihood = -266.88, time = 2.34s
[BernoulliRBM] Iteration 20, pseudo-likelihood = -265.39, time = 2.34s
Training completed.
```

Observations and Conclusions

1. **Data Cleaning:** The initial dataset had missing values, particularly in Customer IDs, which are crucial for user-level analysis. Removing incomplete entries and invalid transactions (cancelled orders, non-positive prices) ensured data integrity.
2. **Preprocessing:** We successfully encoded categorical fields (like Country and StockCode) and scaled numerical features (Quantity and Price). While the core RBM model primarily relies on the binary interaction matrix, these preprocessed features can be integrated into hybrid models or used for detailed customer segmentation analysis.
3. **Data Transformation:** The creation of the binary matrix effectively translated transactional records into customer shopping habits (purchased vs. not purchased). The resulting matrix is highly sparse, which is typical for retail datasets since any individual customer only buys a small fraction of the total available inventory.
4. **Boltzmann Machine Training:** The `BernoulliRBM` learned the underlying probability distribution of the binary user-item matrix. The learned hidden components represent latent customer segments or shopping profiles based on item co-occurrences.
5. **Future Work:** The trained RBM features can be fed into clustering algorithms (like K-Means) to explicitly assign customers to segments, or into classification models if specific target variables are available. Furthermore, adjusting `n_components`, `learning_rate`, and `n_iter` can fine-tune the model's ability to capture complex patterns.